

Aufgabe 3: Lieferkette

Teilnahme-ID: 79418

Bearbeiter/-in dieser Aufgabe:
Fabian Schwarz

April 6, 2026

Contents

1	Lösungsidee	1
2	Umsetzung	2
2.1	Datenstrukturen	2
2.2	Restwege und Streikkosten	3
2.3	Bestimmung der sicheren Route	3
2.4	Rekonstruktion und Ausweichrouten	3
3	Laufzeit	3
4	Werkzeuge	3
5	Beispiele	4
5.1	Beispiel 00	4
5.2	Beispiel 01	4
5.3	Beispiel 02	5
5.4	Beispiel 03	6
5.5	Beispiel 04	6
5.6	Beispiel 05	7
5.7	Beispiel 06	7
5.8	Beispiel 07	7
5.9	Beispiel 08	7
5.10	Beispiel 09	7
5.11	Beispiel 10	8
6	Quellcode	8

1 Lösungsidee

Zu Beginn sollen die Werke und ihre Verbindungen als gewichteter, ungerichteter Graph dargestellt werden, wobei das Gewicht jeder Kante die benötigte Fahrzeit für diese Straße darstellt. Hierfür gruppiert man die Werke in funktionale Gruppen $G_1, \dots, G_n$, mit n als Anzahl der benötigten Arbeitsschritte, wobei G_k alle Knoten (Werke) beinhaltet, die den Arbeitsschritt k erfüllen können. Für jedes Paar von Werken (u, v) wird nun die kürzeste Fahrzeit $d(u, v)$ bestimmt und gespeichert. Diese G_k stellen nun die k Schichten eines Schichtengraphen dar.

Man definiert $S(u)$ als den optimalen Restweg vom Werk u zum Ziel z :

$$S(u) = \begin{cases} d(u, z) & \text{für } u \in G_n \\ \min_{w \in G_{k+1}} \{d(u, w) + S(w)\} & \text{für } u \in G_k : k < n \end{cases}$$

und $R(v)$ als “Streikkosten”, also die übrige Zeit bis zum Ziel z , wenn die Fahrerin gerade vor dem streikenden Werk v steht:

$$R(v) = \min_{u \in G_k \setminus \{v\}} \left(\underbrace{d(v, u)}_{\text{Fahrt zum Ersatz}} + \underbrace{S(u)}_{\text{Restweg ab dort}} \right)$$

Hierbei sind zwei Sonderfälle zu beachten:

- Wenn $|G_k| = 1$ ist $R(v) = \infty$ und somit kann der Weg nicht streik-sicher sein.
- $R(z) = 0$.

Für jeden Arbeitsschritt k betrachtet man nun die frühestmögliche Ankunftszeit unter Betrachtung eines möglichen Streiks $A(v)$ an jedem Werk $v \in G_k$. Für den Start s ist $A(s) = 0$. Um diese zu bestimmen, werden alle Werke $u \in G_{k-1}$ betrachtet:

$$A(v) = \min\{A(u) + d(u, v) \mid u \in G_{k-1} \wedge A(u) + d(u, v) + R(v) \leq T\}$$

wobei T die maximal erlaubte Bearbeitungszeit ist. Damit ist garantiert, dass:

1. Alle vorherigen Werke bereits sicher (i.e. in T erreichbar) sind, da $A(u)$ schon geprüft wurde,
2. wenn genau bei v gestreikt wird, der Weg dennoch sicher ist und
3. der schnellste Pfad ausgewählt wird.

Sobald man bei z ankommt, wird geprüft, ob $A(z) \leq T$. Falls ja, existiert ein Pfad, der jeden möglichen Streik in T umgehen kann.

Um die Pläne auszugeben, werden während der Berechnung Pointer auf die jeweiligen Optima gespeichert:

- Die Normalroute wird durch das Speichern des jeweils optimalen Vorgängers $u \in G_{k-1}$ während der Berechnung von $A(v)$ ermittelt. Sobald das Ziel z erreicht ist, lässt sich die gesamte Strecke durch einfaches rückwärtiges Verfolgen dieser Pointer bis zum Startpunkt s rekonstruieren.
- Für jedes Werk v_i , das Teil dieser Normalroute ist, wird zusätzlich ein individueller Alternativplan bereitgestellt. Sollte vor Ort ein Streik an v_i festgestellt werden, steuert die Fahrerin zunächst das Ersatzwerk $u \in G_k \setminus \{v_i\}$ an, welches bei der Berechnung von $R(v_i)$ das zeitliche Minimum geliefert hat.
- Von diesem Ersatzwerk u aus folgt die weitere Fahrt einer Kette von Idealschritten bis zum Ziel z . Diese Kette setzt sich aus den jeweiligen Nachfolgern $w \in G_{k+1}$ zusammen, die bereits während der Rückwärtsrechnung von $S(u)$ als lokales Minimum, also als der jeweils nächste optimale Schritt, identifiziert und für jedes Werk hinterlegt wurden.

Wie sich in der Laufzeitberechnung herausstellt, ist dieser Ansatz auch für das größte Beispiel (Nummer 10 mit $N = 3932$ und $M = 26040$) in wenigen Sekunden lösbar, weshalb keine weiteren Lösungsansätze betrachten wurden.

2 Umsetzung

2.1 Datenstrukturen

Die Straßenkarte wird als ungerichteter Graph mit der Fahrzeit als Kantengewicht eingelesen. Bereits jetzt werden die Werke nach ihrem Arbeitsschritt in die Gruppen G_k gruppiert.

Für jeden Interessenspunkt, also den Startpunkt S und alle Werke, wird ein Single-Source Shortest Path Dijkstra¹ ausgeführt. Dieser ist in der verwendeten *petgraph* Bibliothek bereits optimal (unter Verwendung einer priority-queue) implementiert. Die Ergebnisse werden in einer geschachtelten `HashMap` `all_dists` gespeichert, um in $O(1)$ die Distanz zwischen zwei beliebigen Werken u, v als `all_dists[u][v]` abfragen zu können.

¹Raman, R. (1997). Recent results on the single-source shortest paths problem. *ACM SIGACT News*, 28(2), 81–87. <https://doi.org/10.1145/261342.261352>

2.2 Restwege und Streikkosten

Nun wird jeder Arbeitsschritt k von n bis 1 rückwärts durchlaufen. Für jedes Werk wird der optimale Restweg zum Ziel berechnet und gespeichert. Gleichzeitig wird für jedes Werk v ein `backup_node`, also das beste Ausweichwerk im Fall eines Streiks, bestimmt, und daraus die resultierenden Streikkosten $R(v)$.

2.3 Bestimmung der sicheren Route

Als nächstes werden in einem Vorwärtsdurchlauf für jedes Werk v in Schicht k alle Vorgängerknoten u aus Schicht $k-1$ geprüft. Ein Übergang wird nur gespeichert, wenn er sicher ist, also $A(u)+d(u,v)+R(v) \leq T$ erfüllt ist. Dies garantiert, dass bei einem Streik an egal welchem Werk v die Frist nie überschritten wird. Ist dies nicht erfüllbar, scheitert der Algorithmus und es gibt keine vollkommen sichere Route.

2.4 Rekonstruktion und Ausweichrouten

Nachdem die sichere Erreichbarkeit des Ziels überprüft wurde, also $A(z) \leq T$, werden die Pfade zurückverfolgt. Ausgehend von einer vorhin gespeicherten Map `prev_secure` wird schrittweise für jedes Werk der optimale Vorgänger bestimmt, bis der Startpunkt erreicht ist.

Für jedes Werk v der Hauptroute wird ein individueller Plan erstellt. Hierbei wird der A*-Algorithmus² aus `petgraph` benutzt, der den konkreten Pfad von Werk v zur vorhin hinterlegten `backup_node` bestimmt. Ab hier wird die Route durch die Kette der vorhin hinterlegten nächstbesten Knoten berechnet.

Die Ergebnisse werden folgend ausgegeben und visualisiert.

3 Laufzeit

Um die kürzesten Wege zwischen allen Paaren von Werken zu berechnen wird der Dijkstra-Algorithmus verwendet, der N mal ausgeführt werden muss, für N Werke, M Straßen und n Arbeitsschritte. Der Graph scheint recht dünn, mit nur durchschnittlich 7 Straßen pro Werk im größten Beispiel (Beispiel 10), weshalb unter der verwendeten Implementierung von einer Laufzeitkomplexität von $O(N(M + N \log N))$ ausgegangen werden kann.³

Bei der Berechnung von $S(u)$ werden für jedes Werk in G_k alle Werke in G_{k+1} betrachtet. Die Anzahl an Vergleichen ist somit durch die Anzahl der Werk-Paare begrenzt, womit $S(u) \in O(N^2)$. Aus dem selben Grund ist $R(v) \in O(N^2)$ für den Worst-Case, dass alle Werke in die selbe Gruppe G_k fallen.

Analog zu der Bestimmung der Komplexität von $S(u)$ ist auch $A(v) \in O(N^2)$.

Die Gesamtlaufzeit bestimmt sich also auf $O(NM + N^2 \log N)$.

Für das größte Beispiel mit $N = 3932$ und $M = 26040$ lässt sich die Belastung also auf circa $300 * 10^6$ Operationen für den Dijkstra-Algorithmus und $16 * 10^6$ Operationen für die S , R und A Tabellen bestimmen, womit sich die Gesamtlaufzeit selbst ohne Multithreading im Bereich weniger Sekunden halten sollte⁴.

4 Werkzeuge

Der Code wurde vollständig in Rust 1.93.1 (01f6ddf75 2026-02-11) geschrieben.

Das `petgraph` Crate wurde für eine optimierte Implementierung von Graphen verwendet. Es verwaltet die Lifetimes von Knoten und Kanten intern, was einigen Boilerplate-Code erspart. Die mitgelieferte Implementierung des Dijkstra-Algorithmus folgt dem min-heap priority queue Ansatz aus Fredman & Tarjan (1987)⁵. Ebenfalls wird die Implementierung des A*-Algorithmus mit der Heuristik 0 verwendet, um am Ende das Backtracking auszuführen, da der eingebaute Dijkstra-Algorithmus die Pfade nicht zurückgibt.

²Die Heuristik ist hier 0, was heißt, dass eigentlich einfach nur der Dijkstra-Algorithmus angewandt wird. Unterschied bei diesem Methodenaufruf ist, dass so der konkrete Pfad zurückgegeben wird, und nicht nur dessen Länge; das ist somit bibliotheksbedingt.

³Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3), 596–615. <https://doi.org/10.1145/28869.28874>

⁴8 x 11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz

⁵Siehe Fußnote 3.

5 Beispiele

5.1 Beispiel 00

 MOEGLICH

15

9

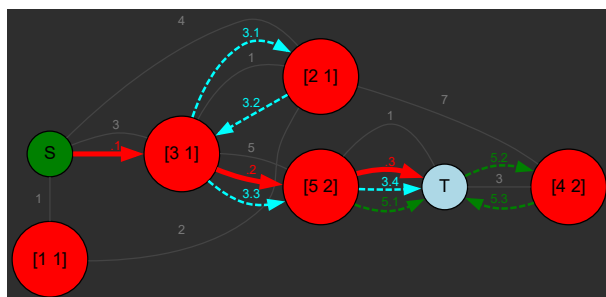
S [3 1] [5 2] T

11

3 [2 1] 3 [5 2] T

15

5 T [4 2] T



Die grauen Linien stellen hier die Straßen mit entsprechenden Gewichten (Fahrzeiten) dar. Die Pfeile .1, .2, ... zeigen den Standardpfad. Die Pfeile 3.1, 3.2, ... zeigen den Ersatzpfad, für den Fall dass Werk 3 bestreikt wird, und so weiter. Werke sind hier als [N, K] dargestellt, wobei N die Nummer des Werks und K der dort machbare Verarbeitungsschritt ist. Diese Pfade sind offensichtlich optimal.

5.2 Beispiel 01

 MOEGLICH

40

26

S [1 1] [4 2] [7 3] T

40

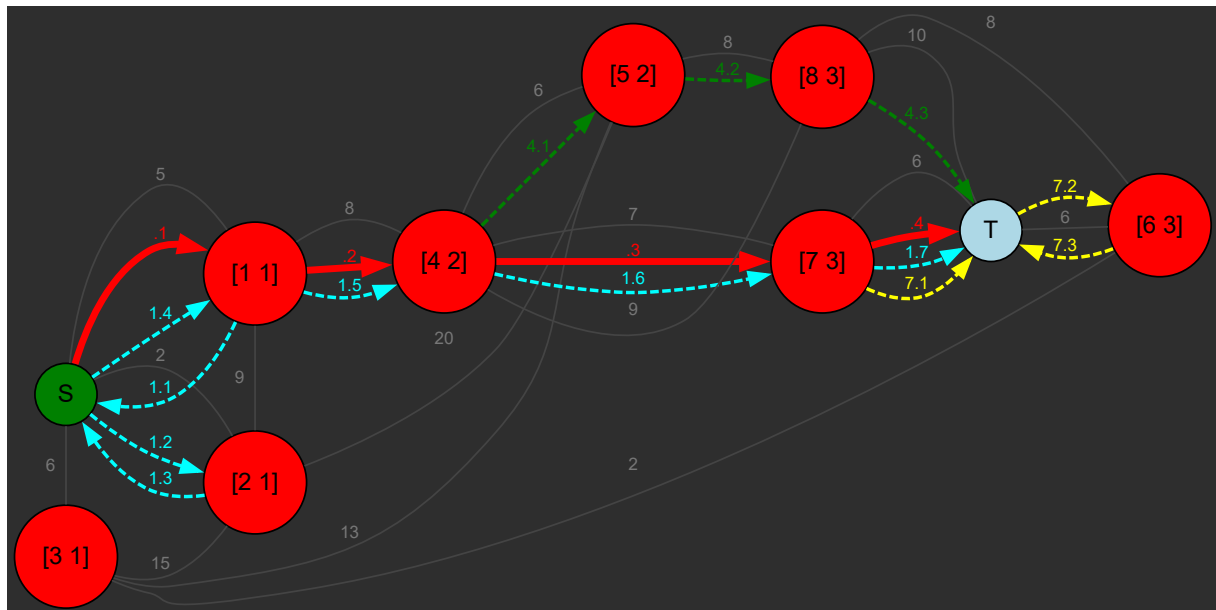
1 S [2 1] S 1 [4 2] [7 3] T

37

4 [5 2] [8 3] T

38

7 T [6 3] T



Auch diese Pfade scheinen optimal.

5.3 Beispiel 02

MOEGLICH

188

122

S [1 1] [2 2] [4 3] [10 4] [7 5] T

138

1 6 [5 1] [3 2] [4 3] 2 13 7 [10 4] [7 5] 13 2 S 1 T

157

2 13 7 5 [3 2] [4 3] 2 13 7 [10 4] [7 5] 13 2 S 1 T

188

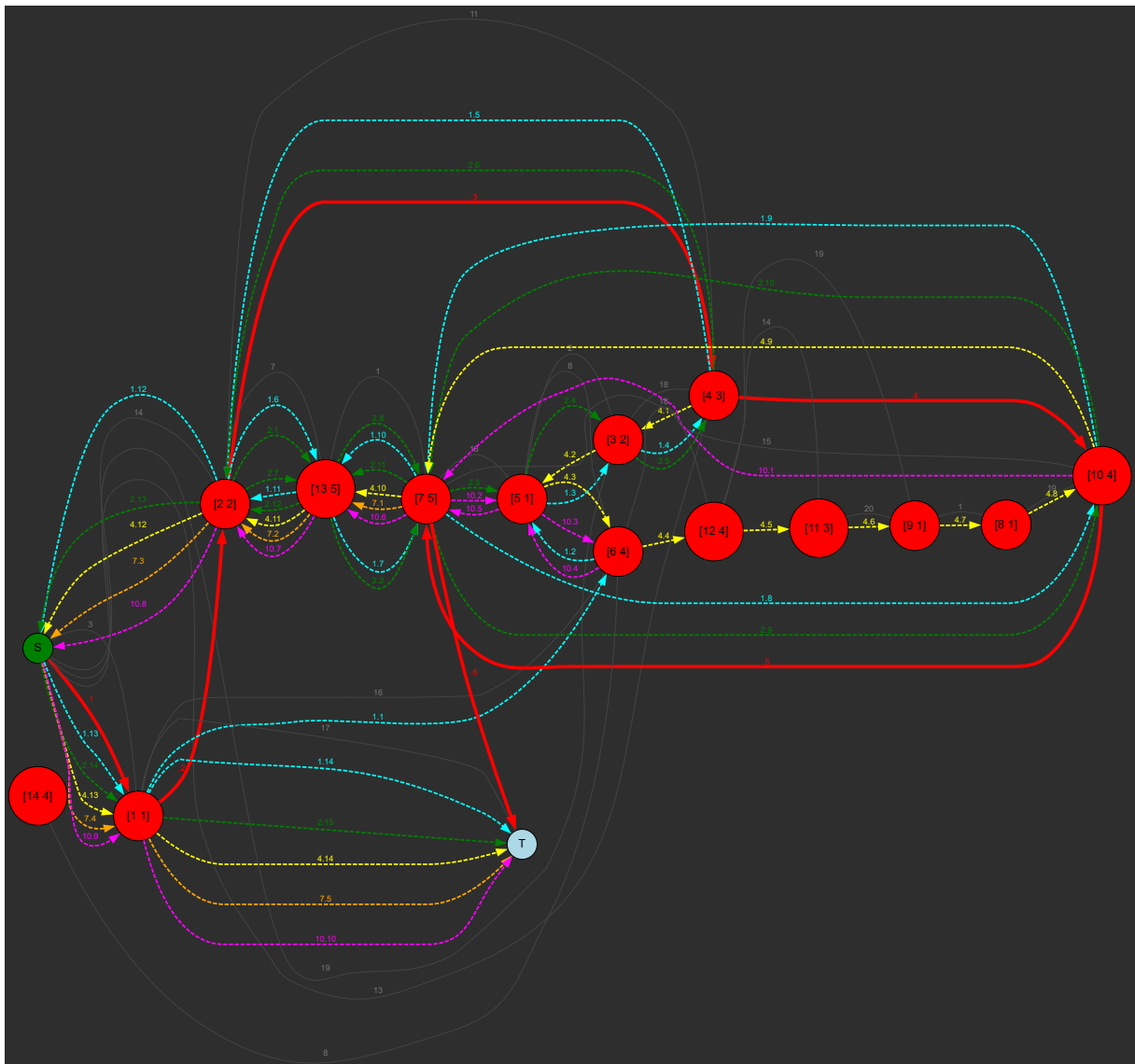
4 3 5 6 12 [11 3] 9 8 [10 4] [7 5] 13 2 S 1 T

174

10 7 5 [6 4] 5 [7 5] 13 2 S 1 T

122

7 [13 5] 2 S 1 T



Hier ist es schwierig zu erkennen, ob tatsächlich optimale Pfade gewählt worden sind. Die wiederholte Verwendung bestimmter Teilpfade auf unterschiedlichen Routen lässt dies jedoch vermuten; beispielsweise verwenden die Ersatzrouten für Werke 1, 2, 4 und 10 alle den Weg zu T über S, anstatt über Werk 7 wie der Standardpfad.

5.4 Beispiel 03

UNMOEGlich

5.5 Beispiel 04

MOEGlich
264
[...]

5.6 Beispiel 05

MOEGLICH
930
[...]

5.7 Beispiel 06

UNMOEGLICH

5.8 Beispiel 07

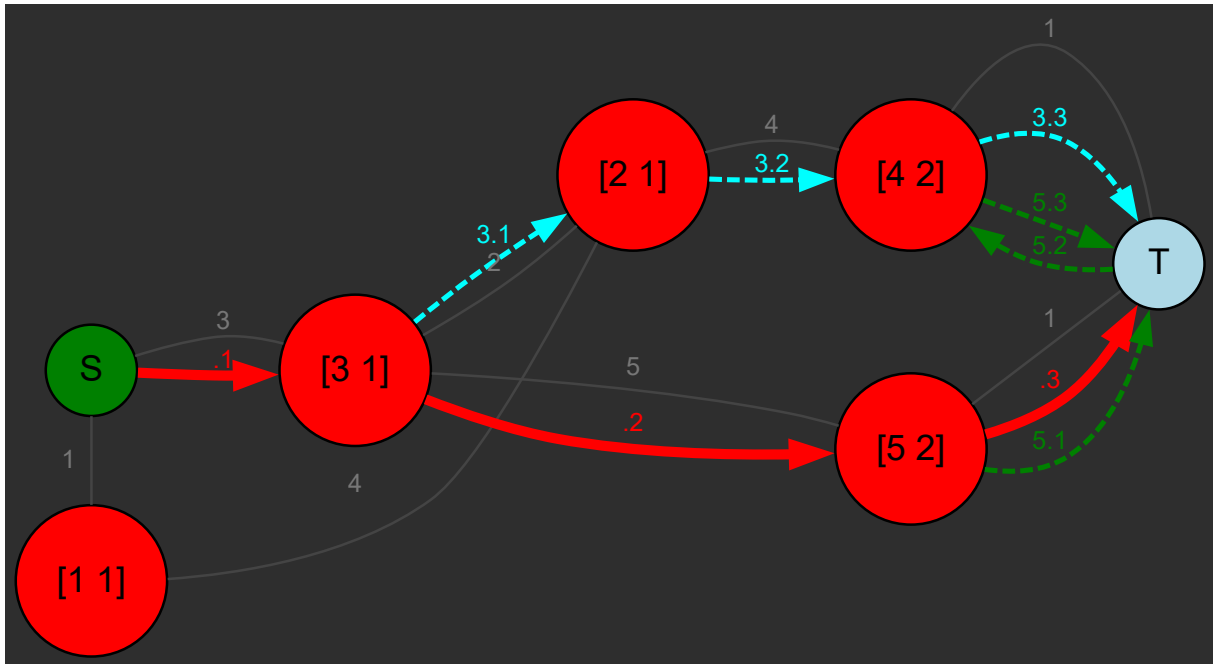
MOEGLICH
675
[...]

5.9 Beispiel 08

MOEGLICH
1333
[...]

5.10 Beispiel 09

MOEGLICH
11
[...]



Auch dieses Beispiel ist recht offensichtlich optimal.

5.11 Beispiel 10

MOEGLICH
1462
[...]

6 Quellcode

Aussparungen sind hier mit `/* [...] */` markiert.

```
/* [...] */
fn solve() {
  /* [...] Einlesen der Datei und Initialisierung des Graphen [...] */

  // Für jeden relevanten Ort (Start und alle Werke) wird einmalig Dijkstra ausgeführt
  // Die Distanzen d(u, v) werden für O(1) Zugriff in einer Map gespeichert
  let mut all_dists = HashMap::new();
  for start in pois {
    let res = petgraph::algo::dijkstra(&graph, loc_to_idx[&start], None, |e| *e.weight());
    all_dists.insert(start, res.into_iter().map(|(i, d)| (graph[i].clone(), d)).collect());
  }

  // Rückwärts DP
  // s_val[u]: Optimaler Restweg von Werk u bis zum Ziel T
  // r_val[v]: Zeit bis Ziel, wenn Werk v bestreikt wird
  for step in (1..=data.count_steps).rev() {
    for &u in &data.grouped_factories[step as usize] {
      if step == data.count_steps {
        s_val.insert(u, all_dists[&u_loc][&LocationType::Goal]);
        next_best.insert(u, LocationType::Goal);
      } else {
        // Suche das Werk w im nächsten Schritt, das den Restweg minimiert
        /* [...] Minimum-Suche über alle w in G_{step+1} [...] */

```



```

        s_val.insert(u, min_dist);
        next_best.insert(u, best_w);
    }
}

// Berechnung des besten Ersatzwerks u in der selben Gruppe
for &v in &data.grouped_factories[step as usize] {
    /* [...] Suche u in G_{step} \ {v}, das d(v, u) + s_val[u] minimiert [...] */
    r_val.insert(v, min_repair_dist);
    backup_node.insert(v, best_backup);
}

// Vorwärts DP
// a_val[v]: Früheste Ankunftszeit bei v, die garantieren kann, dass bei einem
// anschließenden Streik von v das Ziel noch innerhalb von max_time erreicht wird
a_val.insert(LocationType::Start, 0);
for step in 1..=data.count_steps {
    for &v in &data.grouped_factories[step as usize] {
        // Prüfe alle potentiellen Vorgänger u aus der vorherigen Schicht
        for u_l in prev_layer {
            let d = all_dists[&u_l][&v_loc];
            if a_val[&u_l] + d + r_val[&v] <= data.max_time as i32 {
                /* [...] Update a_val[v] und Speicherung des Vorgängers in prev_secure
                [...] */
            }
        }
    }
}

if final_a <= data.max_time as i32 {
    /* [...] Backtracking-Loop [...] */

    // Alternativrouten
    for node in &main_route {
        if let LocationType::Num(id) = node.loc {
            // Pfad zum Ersatzwerk mit Dijkstra (A* mit Heuristik 0)
            let (path_to_backup) = petgraph::algo::astar(&graph, /* [...] */);
            /* [...] Folgen der next_best Kette [...] */
        }
    }
} else {
    /* [...] Unmöglich */
}
}

```
